

HelixMemory

HelixMemory

Ein Gedächtnisgehirn für AI-Agenten - vier erstklassige Motoren, verschmolzen.

Zusammenfassung

HelixMemory ist ein Go-SDK, das vier führende Gedächtnissysteme (Mem0, Cognee, Letta, Graphiti) zu einer einzigen kognitiven Gedächtnis-Engine vereint, die sie parallel durchsucht und die Ergebnisse fusioniert. Es bietet AI-Anwendungen eine einzige dauerhafte, deduplizierte und neu bewertete Gedächtnisebene statt vier voneinander getrennter.

Kurzbeschreibung

HelixMemory ist ein Go-SDK, das Mem0, Cognee, Letta und Graphiti zu einer einheitlichen kognitiven Gedächtnis-Engine für AI-Anwendungen verschmilzt. Es leitet Schreibvorgänge intelligent weiter, durchsucht alle Backends parallel und fusioniert die Ergebnisse in einem dreistufigen Sammel-Deduplizierungs-Neubewertungs-Prozess.

Ausführliche Beschreibung

HelixMemory ist eine einheitliche kognitive Gedächtnis-Engine für AI-Anwendungen, bereitgestellt als Go-SDK (Modul `digital.vasic.helixmemory`, Go 1.25+). Die grundlegende Annahme dabei ist, dass kein einzelnes Gedächtnisprojekt jemals in allen Bereichen führend sein wird – statt also Gedächtnisfunktionen von Grund auf neu zu implementieren und die blinden Flecken eines Projekts zu übernehmen, orchestriert es vier erstklassige Systeme und lässt jedes seine Stärken ausspielen: Mem0 für dynamische Faktenextraktion und Präferenzmanagement, Cognee für semantische Wissensgraphen, die über ECL-Pipelines aufgebaut werden, Letta für eine zustandsbehaftete Agentenlaufzeit mit

editierbaren Gedächtnisblöcken und Rechenoperationen während der Ruhephase sowie Graphiti für einen bitemporalen Wissensgraphen, der Schlussfolgerungen über die zeitliche Veränderung von Fakten ermöglicht.

Eine Fusions-Engine ist es, die diese vier unabhängigen Speicher zu einem einzigen Gehirn vereint. Beim Schreibvorgang wird jedes eingehende Gedächtniselement nach Inhalt klassifiziert und an das am besten geeignete Backend weitergeleitet. Beim Lesevorgang wird eine Abfrage parallel an alle Backends gesendet, und die Rohstreffer fließen in eine dreistufige Fusions-Pipeline – Sammlung, Deduplizierung und quellenübergreifende Neubewertung –, sodass der Aufrufer nicht vier unübersichtliche, sich überschneidende Ergebnismengen erhält, sondern eine saubere, bewertete Antwort. Circuit Breaker umschließen jedes Backend, um eine elegante Degradation zu ermöglichen: Fällt eine Engine aus, löst ihr Breaker aus, und die verbleibenden Backends bleiben funktionsfähig, statt die gesamte Gedächtnisebene mitzureißen. Da die Engine eine nahtlos integrierbare MemoryStore-Schnittstelle implementiert, lässt sie sich als direkter Ersatz für einen einfachen Gedächtnisanbieter einsetzen – ohne dass der Aufrufer umstrukturiert werden muss – und Prometheus-Metriken machen die internen Abläufe von Weiterleitung und Fusion vollständig beobachtbar.

HelixMemory wurde als Gedächtnisebene für HelixAgent entwickelt, das umfassendere Helix-AI-Ensemble, und überträgt die familientypische Anti-Bluff-Testdisziplin auf den Gedächtnisbereich: Ein in den Prozess integrierter Challenge-Runner prüft reale Produktionscode-Pfade – Weiterleitung, Fusion, Übersetzer, Circuit Breaker –, während ein gekoppelter Mutations-Wrapper gezielt Invarianten verändert, um zu beweisen, dass die Tests tatsächlich fehlschlagen, wenn die Logik fehlerhaft ist. Ein grünes Testergebnis bedeutet also etwas.

Warum wir es entwickelt haben

AI-Agenten benötigen langlebige, hochwertige Speicherlösungen, doch das Ökosystem ist fragmentiert – jedes Speicherprojekt (Mem0, Cognee, Letta, Graphiti) glänzt in einem Bereich und hat in anderen Schwächen. HelixMemory wurde entwickelt, um HelixAgent eine einheitliche Speicheroberfläche zu bieten, die ihre Stärken vereint, ohne eine Bindung an eines der Systeme zu erzwingen.

Warum es die Spielregeln ändert

Es beendet die erzwungene Entscheidung. Vier Speichersysteme, die normalerweise um denselben Platz konkurrieren, werden zu komplementären Backends hinter einer einzigen Schnittstelle – sodass eine Anwendung dynamische Faktenextraktion, semantische Wissensgraphen, zustandsbehafteten Agentenspeicher und bitemporale Logik *gleichzeitig* nutzen kann, während Deduplizierung und quellenübergreifende Neurangfolgen automatisch abgewickelt werden. Was zuvor unpraktikabel war, wird nun möglich: Die Frage „Welches Speichersystem übernehmen wir?“ ist kein echtes Dilemma mehr. HelixMemory ermöglicht es, alle Stärken der Systeme auf einmal zu nutzen – hinter einem einfach integrierbaren MemoryStore – ohne die Schwächen eines einzelnen Systems zu erben oder sich auf eine langfristige Bindung festzulegen.

Was innovativ ist

- **Mehrfach-Backend-Fusion** (Sammeln → Deduplizieren → quellenübergreifende Neurangfolge), die einen einzigen, sortierten Ergebnissatz liefert, statt den Aufrufer an ein einzelnes Speichersystem zu ketten.
- **Intelligentes Schreib-Routing**, das jeden Speichereintrag nach Inhalt klassifiziert und an das am besten geeignete Backend weiterleitet, sodass die richtigen Daten im richtigen Speicher landen.
- **Graceful Degradation** durch Backend-spezifische Schutzschalter – ein ausgefallenes System wird isoliert, ist aber nicht fatal, und die übrigen Backends bleiben weiterhin verfügbar.
- **Schlafzeit-Compute-Konsolidierung** (via Letta), die Speicherinhalte in Leerlaufphasen neu aufbereitet, statt nur zur Abfragezeit zu arbeiten.
- **Anti-Bluff-Verifizierung**: ein Challenge-Runner für echten Produktionscode, kombiniert mit einem Mutations-Wrapper, der bei Verletzung einer Invarianten fehlschlagen *muss* – um zu beweisen, dass die Testhürde ein echter Check ist und keine Tautologie.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Vier heterogene Backends zu einem kohärenten Ergebnissatz vereinen** – jedes System liefert Speicherinhalte in eigener Struktur, und ein naives Zusammenführen führt zu Duplikaten und nicht vergleichbaren Rangfolgen. Gelöst durch eine typisierte Fusions-Engine, die Daten aus allen Quellen

sammelt, Überschneidungen dedupliziert und alles auf einer gemeinsamen Basis neu bewertet, wobei die Fusionszähl-Invariante in Tests überprüft wird, damit die Zusammenführung keine Ergebnisse stillschweigend verliert oder doppelt zählt.

- **Verfügbarkeit auch bei Ausfall eines Backends sicherstellen** – ein nicht erreichbares Speichersystem darf die gesamte Schicht nicht blockieren. Gelöst durch Backend-spezifische Schutzschalter, die nach einem festgelegten Fehler-Schwellenwert von „geschlossen“ auf „offen“ (nach Ausfall) und nach einer Wartezeit auf „halb-offen“ wechseln, das defekte Backend isolieren und die Anfragen an die funktionierenden Systeme weiterleiten, bis es sich erholt.
- **Beweisen, dass die Speicherlogik tatsächlich funktioniert - und nicht nur kompiliert** – eine grüne Testsuite ist wertlos, wenn die Tests nicht fehlschlagen können. Gelöst durch einen In-Process-Challenge-Runner, der echten Produktionscode (Routing, Fusion, Übersetzer, Schutzschalter) ausführt, sowie einen Mutations-Wrapper, der Invarianten gezielt verändert und verlangt, dass die Tests rot werden – sodass die Testhürde nachweislich keine Tautologie ist.

Tech-Stack

- **Go (1.25+)** — der einzelne SDK und die Laufzeitumgebung; ausgewählt, weil das parallele Auslesen über vier Backends ein Nebenläufigkeitsproblem darstellt und Gos Goroutinen dies kostengünstig ermöglichen, während seine Schnittstellentypen dem gesamten System eine saubere Nahtstelle (MemoryStore) bieten, auf die sich Aufrufer verlassen können.
- **Mem0** — das Backend für dynamische Faktenextraktion und Präferenzmanagement; genutzt für den „welche Präferenzen hat dieser Nutzer tatsächlich / welche Fakten sind zutage getreten“-Ausschnitt des Gedächtnisses.
- **Cognee** — das semantische Wissensgraph-Backend, aufgebaut auf ECL-Pipelines; dient zur Speicherung strukturierten, verknüpften Wissens statt flacher Fakten.
- **Letta** — das zustandsbehaftete Agenten-Laufzeit-Backend mit editierbaren Gedächtnisblöcken und Berechnungen während der Ruhephasen; eingesetzt, wo Gedächtnisinhalte als aktiver Agentenzustand bestehen bleiben und in Leerlaufzeiten konsolidiert werden müssen.
- **Graphiti** — das bitemporale Wissensgraph-Backend; genutzt, um Schlussfolgerungen über die Veränderung von Fakten und Beziehungen im Zeitverlauf zu ziehen, nicht nur über ihren aktuellen Wert.
- **PostgreSQL + Neo4j + Redis** — die realen Datenspeicher, auf denen die Backends laufen und die für echte

Integrationstests über `make infra-start` hochgefahren werden, sodass die Testsuite echte Infrastruktur statt Mocks nutzt.

- **Prometheus** — Metriken und Observability, eingebunden über die Fusionspipeline, sodass Routing- und Fusionsverhalten im Produktivbetrieb messbar sind und kein Blackbox-System darstellen.
- **i18n-Übersetzungsschnittstelle** — eine mit Namensraum (`helixmemory_`) versehene Zeichenketten-Oberfläche, die erhalten bleibt, damit jede zukünftige nutzerorientierte Schicht ohne nachträgliche Anpassungen des Kernsystems lokalisiert werden kann.

Status & Transparenzhinweise

- **Status: Beta.** Funktionsfähiger SDK; entwickelt als Gedächtnisebene für HelixAgent.
- **Lizenz: Unklar.** Keine LICENSE-Datei wurde über den GitHub-API-Mechanismus erkannt — UNBESTÄTIGT / nicht deklariert.
- Der Anzeigename „HelixMemory“ verweist auf das Repository `memory`. Die im README genannten Genauigkeitsangaben stammen von den Angaben der vorgelagerten Anbieter, nicht von HelixMemory-Messungen, und wurden hier weggelassen.

Prioritätsstufe: Helix-primär.